

igv-webapp ↔ InfiniteTable Interaction Diagrams

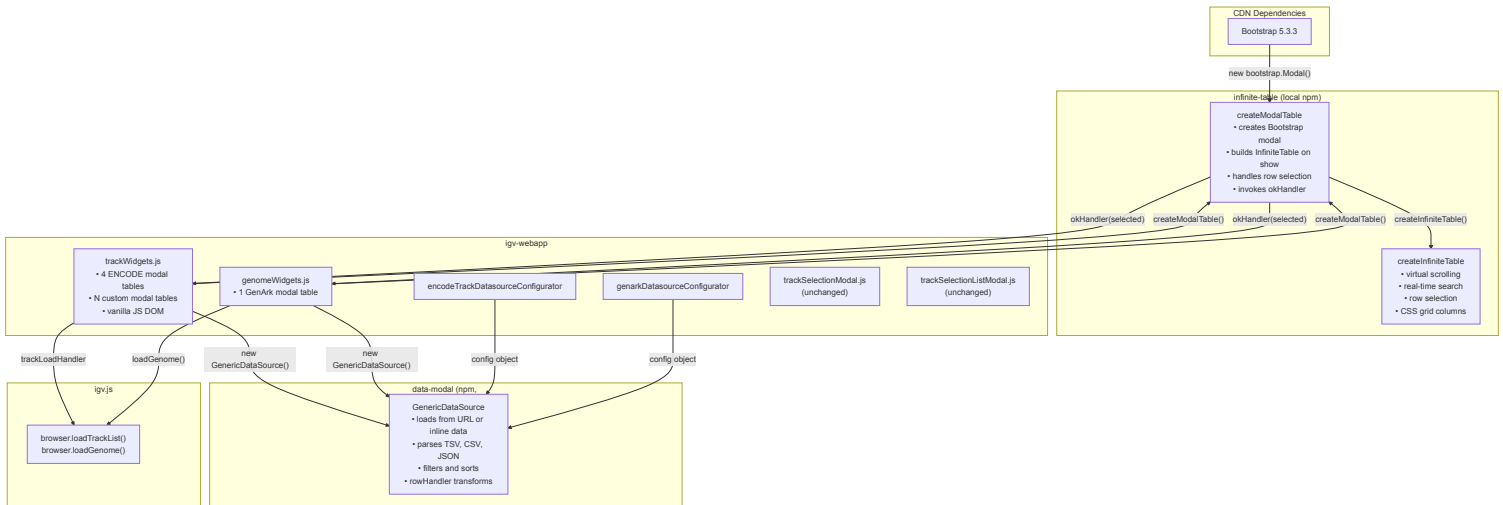
How InfiniteTable replaces jQuery DataTables in igv-webapp, eliminating jQuery and all DataTables dependencies. InfiniteTable is a zero-dependency, virtual-scrolling table component composed from four functional modules.

1. Architecture Overview

Two completely separate modal systems coexist for track/genome selection. InfiniteTable replaces the DataTables-based system; the other two are unchanged:

Modal System	Library	Used For	Selection
createModalTable (InfiniteTable)	Zero-dependency virtual scroller	ENCODE tracks, GenArk genomes, custom-data-modal registries (e.g. 4DN)	Single or multi row click
TrackSelectionModal	Pure Bootstrap 5	Track hubs, genome track registries with groups	Checkboxes with shift-select
TrackSelectionListModal	Pure Bootstrap 5	Flat track lists (no groups)	HTML <code><select multiple></code>

InfiniteTable has **no external dependencies** — no jQuery, no DataTables, no third-party libraries.



2. Dependency Chain

```
index.html (CDN)
├─ Bootstrap 5.3.3 ─────────── modal chrome (used by everything)
└─ (no jQuery, no DataTables)

index.html (local CSS)
├─ node_modules/infinite-table/css/infinite-table.css
```

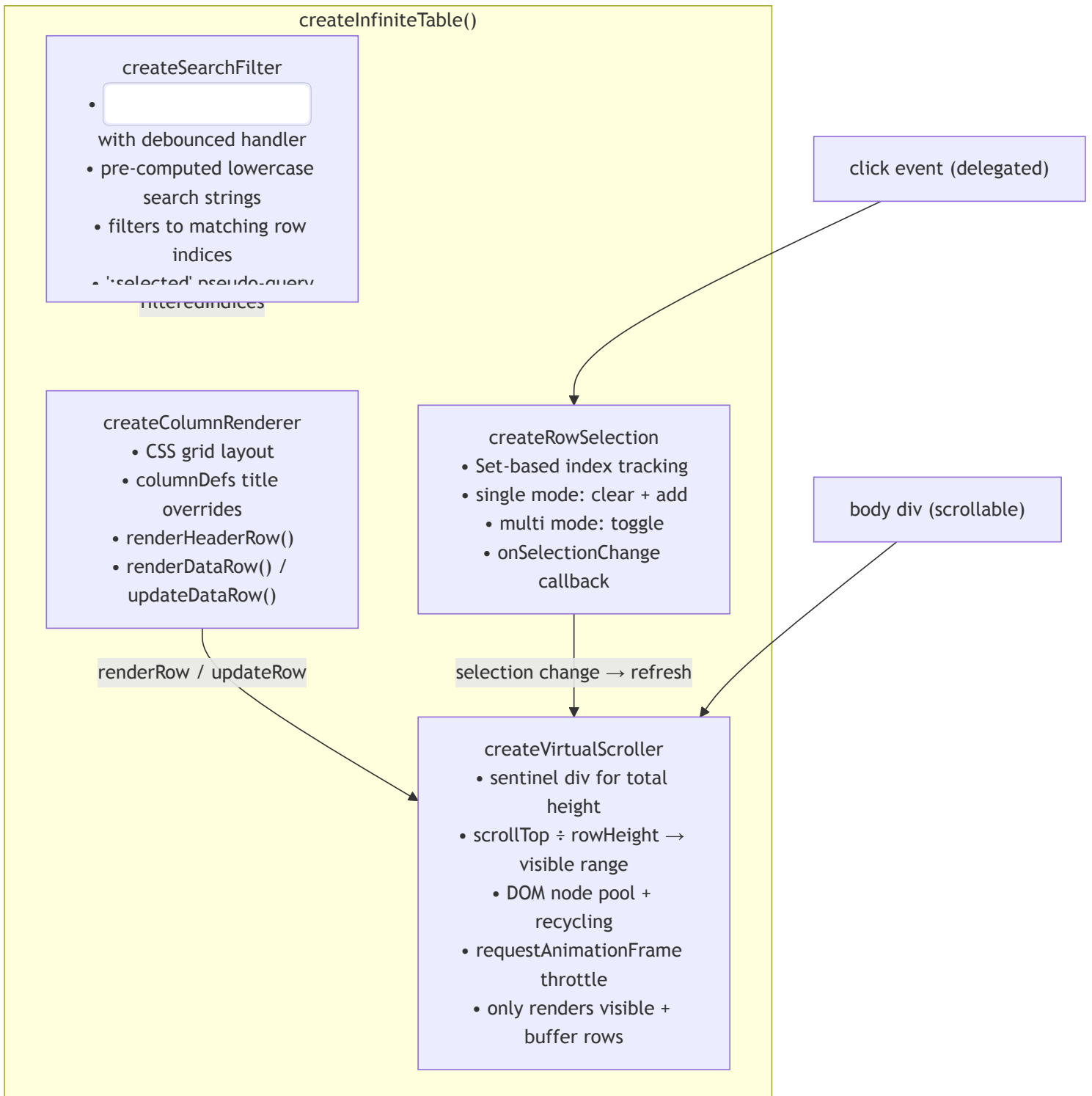
```
package.json (npm)
└─ infinite-table (file:../InfiniteTable)

|   └─ src/modalTable.js ───────── Bootstrap modal wrapper for InfiniteTable
|   └─ src/infiniteTable.js ────── virtual-scrolling table engine
|   └─ src/virtualScroller.js ──── 0(1) scroll position → row index, DOM recycling
|   └─ src/searchFilter.js ────── debounced real-time text search
|   └─ src/rowSelection.js ────── single/multi selection state via Set
|   └─ src/columnRenderer.js ───── CSS grid header/data row rendering
|   └─ src/domUtils.js ────────── minimal DOM helpers
|
└─ data-modal v1.6.2 (temporary - only GenericDataSource used)
    └─ src/genericDataSource.js ─── data loading / parsing / transforms
    └─ src/dataWrapper.js ───────── line-by-line string iteration
    └─ src/stringLoader.js ─────── default fetch()-based HTTP loader
```

jQuery is **completely eliminated** — no CDN script tag, no `$()` calls in trackWidgets.js. DataTables CSS and JS CDN links are removed.

3. InfiniteTable Internal Composition

InfiniteTable is built from four composable functional modules, each created via factory functions (no classes):



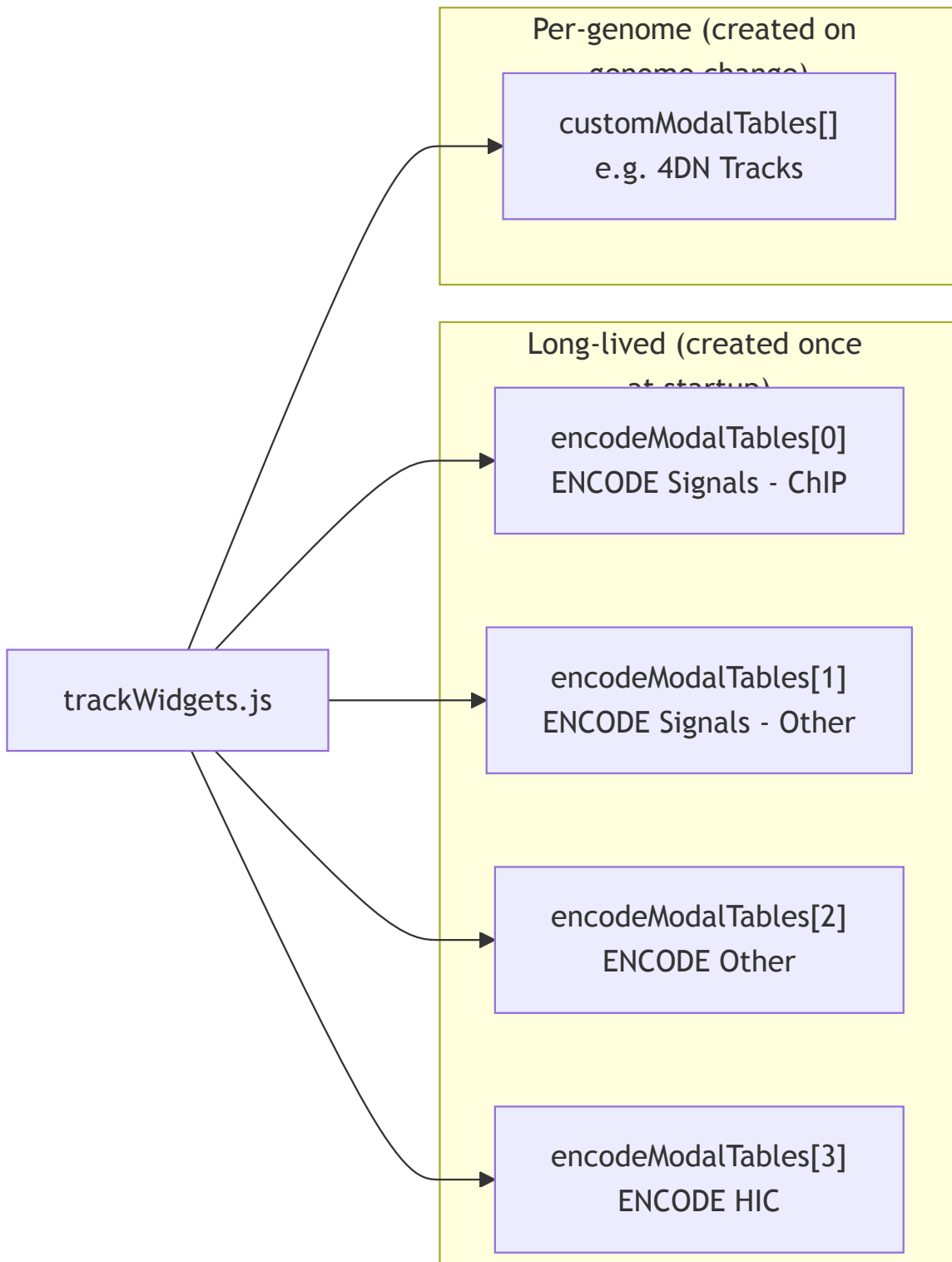
Component Responsibility Matrix

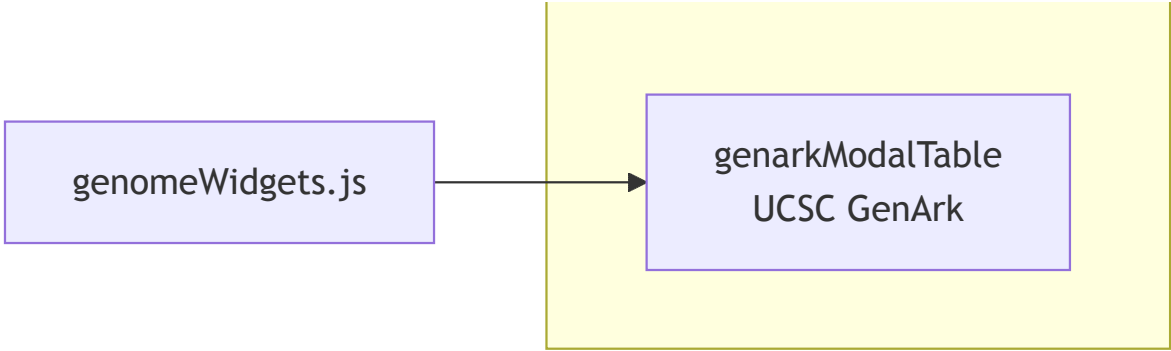
Component	State Owned	DOM Created	Key Methods
	<code>allData[]</code>		

searchFilter	<code>update() ,</code> <code>searchStrings[] ,</code> <code>filteredIndices[]</code>	<code><input></code> search box	<code>setData() , applyFilter() ,</code> <code>getFilteredIndices()</code>
columnRenderer	<code>columns , columnDefs ,</code> <code>gridTemplate</code>	Header row, data row cells	<code>renderHeaderRow() ,</code> <code>renderDataRow() ,</code> <code>updateDataRow()</code>
rowSelection	<code>selected: Set<index></code>	(none — purely logical)	<code>handleRowClick() , isSelected() ,</code> <code>getSelectedIndices()</code>
virtualScroller	<code>rowCount</code> , scroll position, DOM pool	Sentinel div, visible rows container	<code>setRowCount() , scrollToTop() ,</code> <code>refresh()</code>

4. ModalTable Instances — Who Creates What

Same instance lifecycle as before. Five to N instances; four long-lived ENCODE, one long-lived GenArk, zero or more ephemeral custom. The only change is `new ModalTable()` → `createModalTable()` .



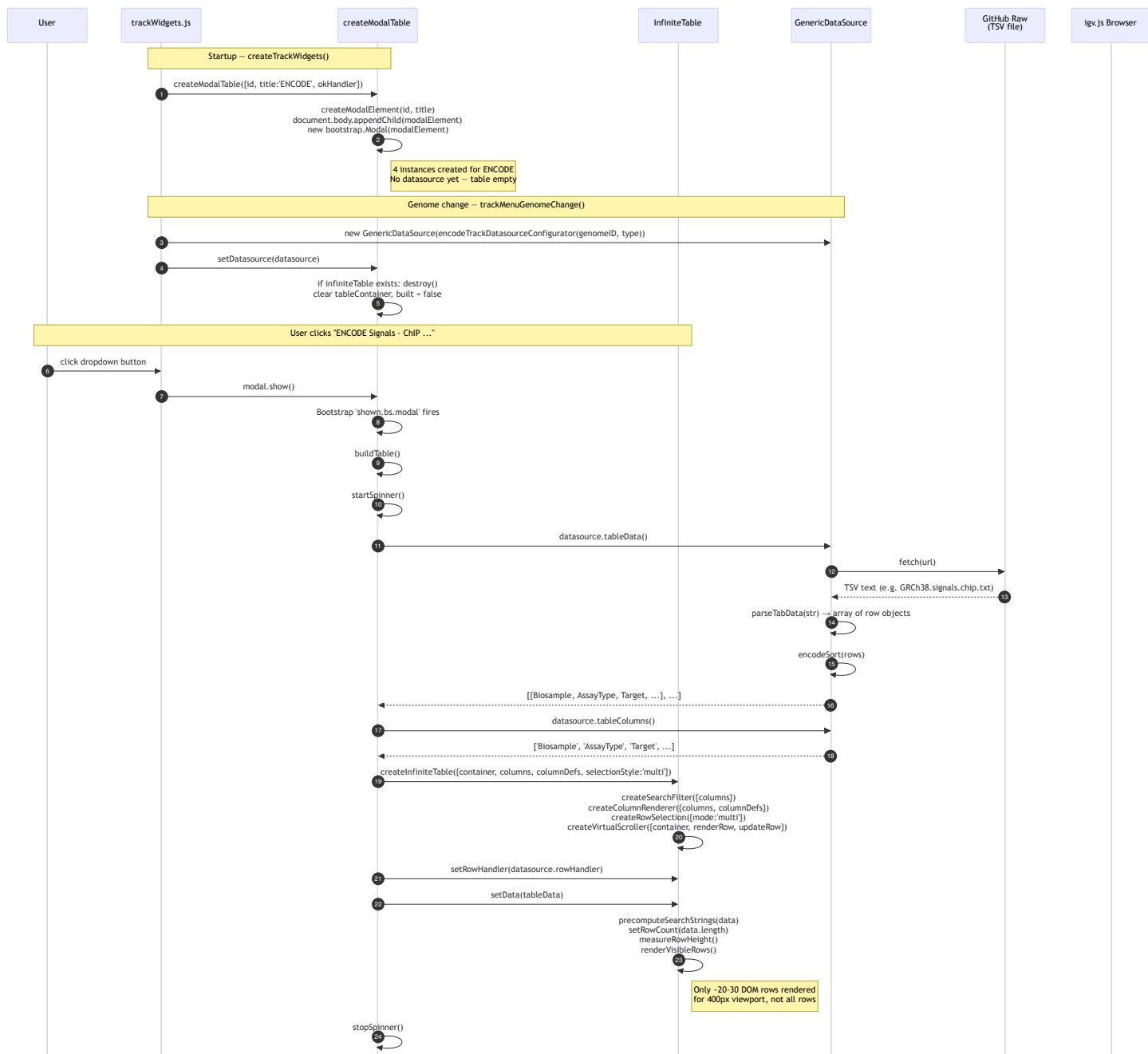


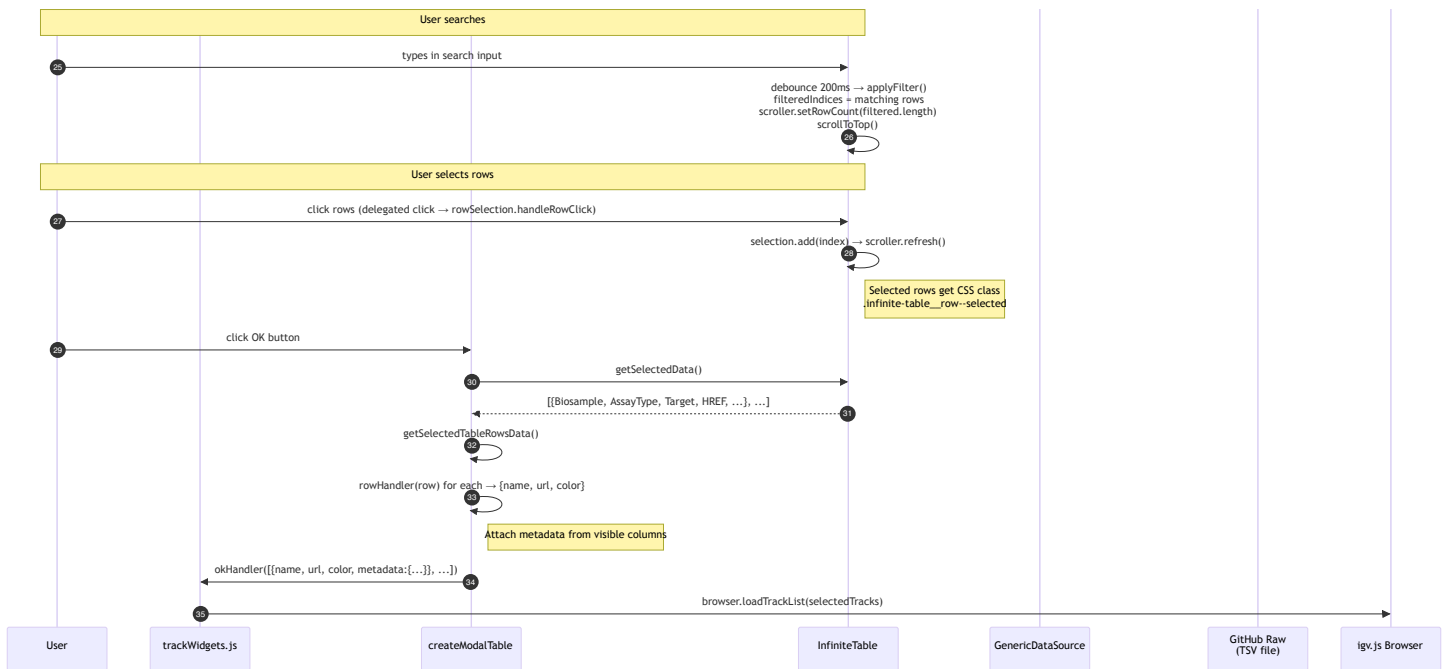
Lifecycle Details

Instance	Created	DataSource Set	Destroyed
ENCODE (x4)	<code>createTrackWidgets()</code> via <code>createModalTable()</code>	<code>trackMenuGenomeChange()</code> via <code>setDatasource()</code>	Never (reused)
GenArk (x1)	<code>createGenomeWidgets()</code> via <code>createModalTable()</code>	Immediately after creation via <code>setDatasource()</code>	Never
Custom (xN)	<code>prepRegistryConfig()</code> via <code>createModalTable()</code>	At creation (datasource in config)	<code>trackMenuGenomeChange()</code> calls <code>remove()</code>

5. ENCODE Track Selection — Full Sequence

The most common interaction. User browses ENCODE tracks with real-time search and virtual scrolling.





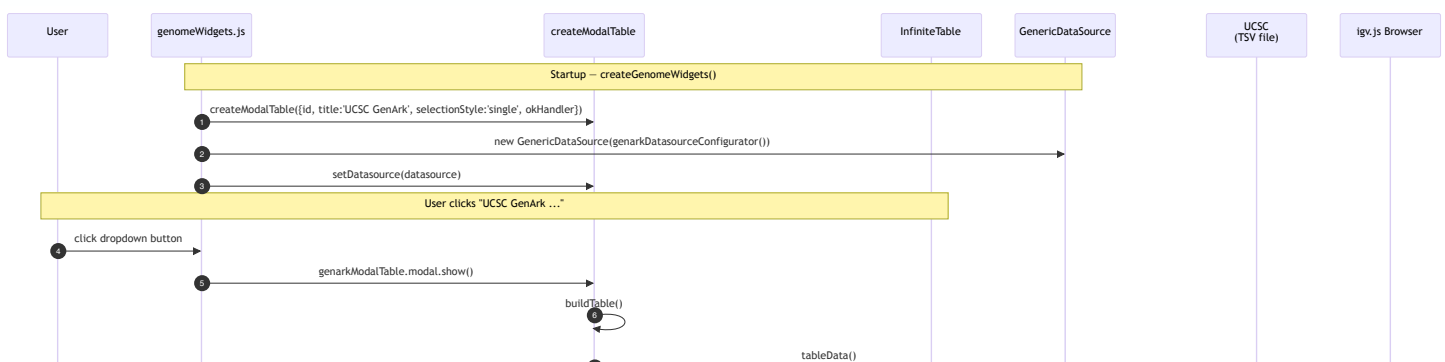
InfiniteTable Configuration Generated

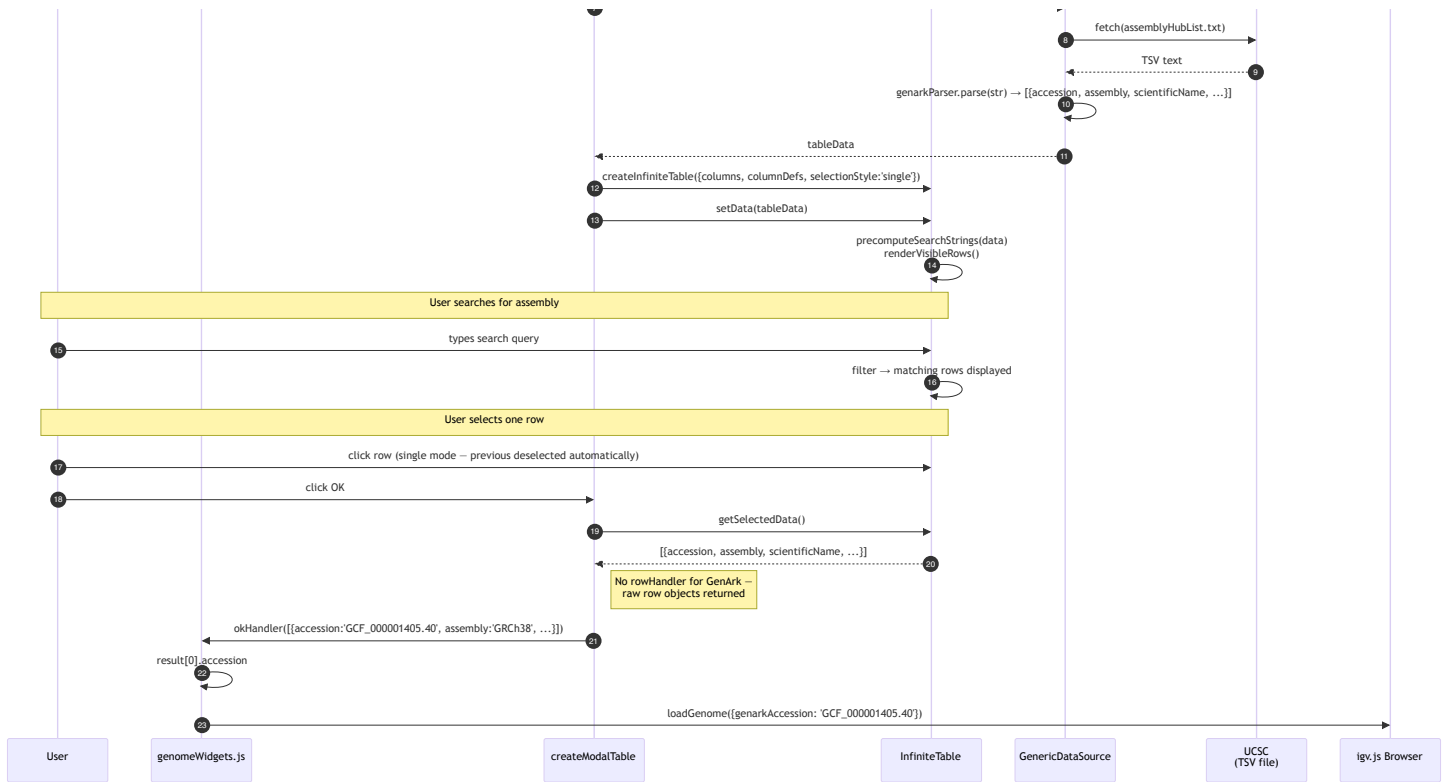
```

createInfiniteTable({
  container: tableContainer,           // div inside modal body
  columns: ['Biosample', 'AssayType', 'Target', 'OutputType',
            'Format', 'Lab', 'Accession', 'Experiment', 'BioRep', 'TechRep'],
  columnDefs: {
    AssayType: { title: 'Assay Type' },
    OutputType: { title: 'Output Type' },
    BioRep: { title: 'Bio Rep' },
    TechRep: { title: 'Tech Rep' }
  },
  selectionStyle: 'multi'              // toggle selection on click
})
// Followed by:
//   infiniteTable.setRowHandler(datasource.rowHandler)
//   infiniteTable.setData(tableData)
  
```

6. GenArk Genome Selection — Full Sequence

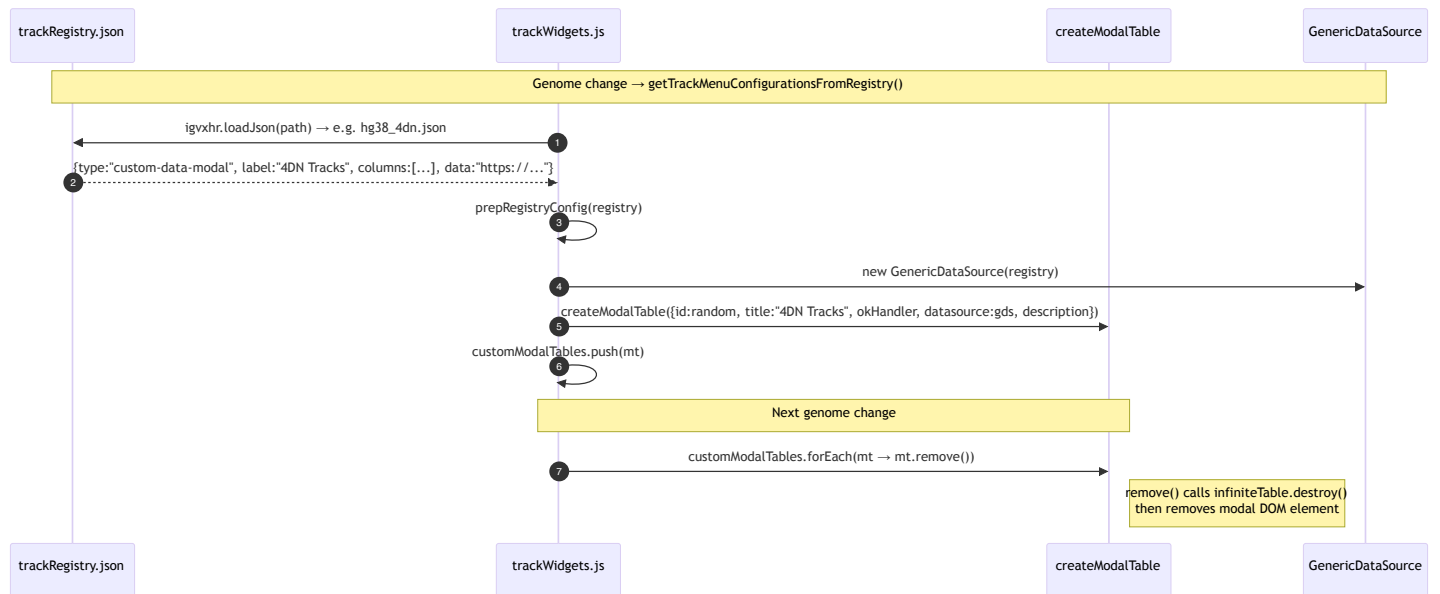
Single-select variant. User picks one genome assembly from the UCSC GenArk catalog.





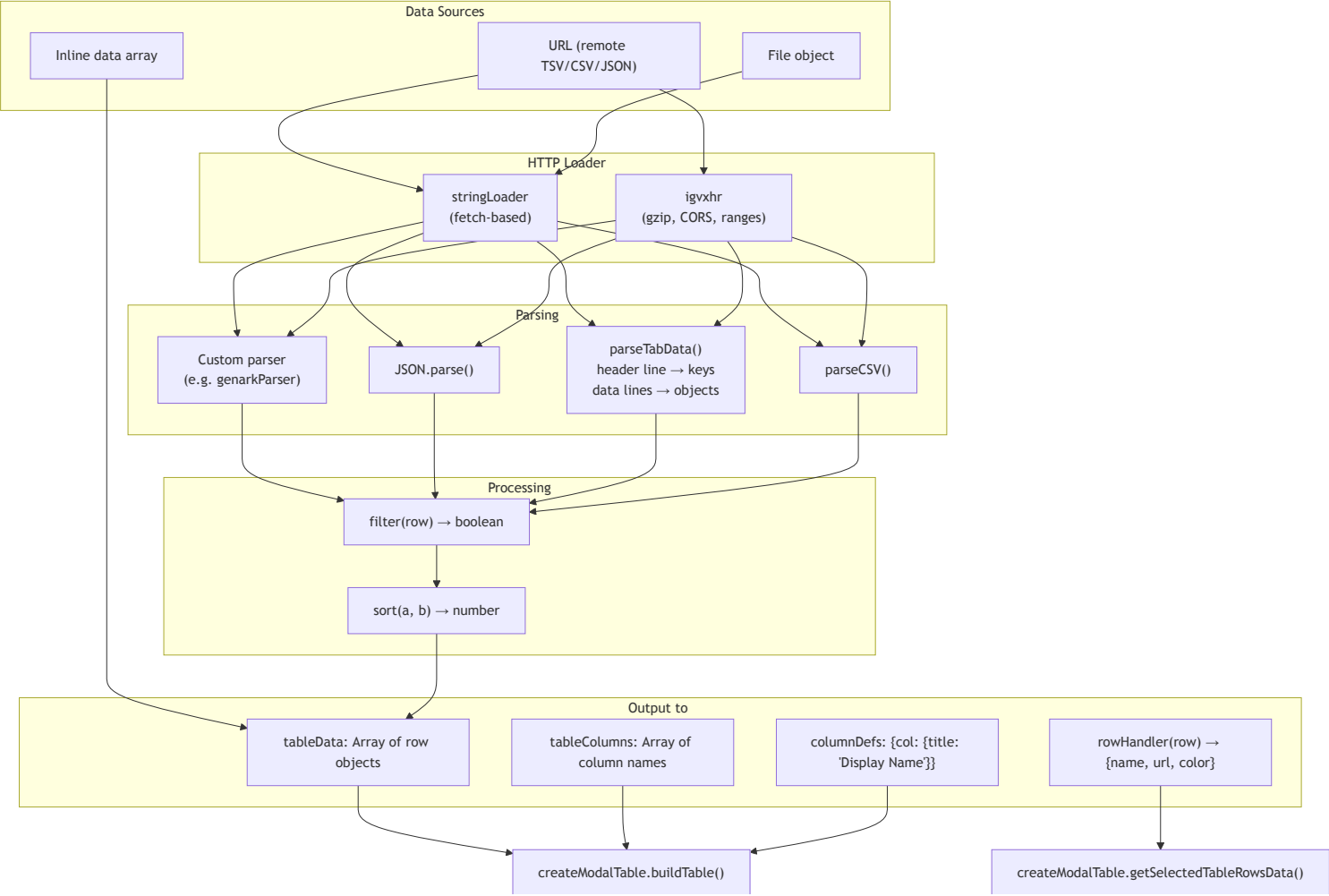
7. Custom Data Modal — Registry-Driven

Unchanged flow — only the table constructor call differs:

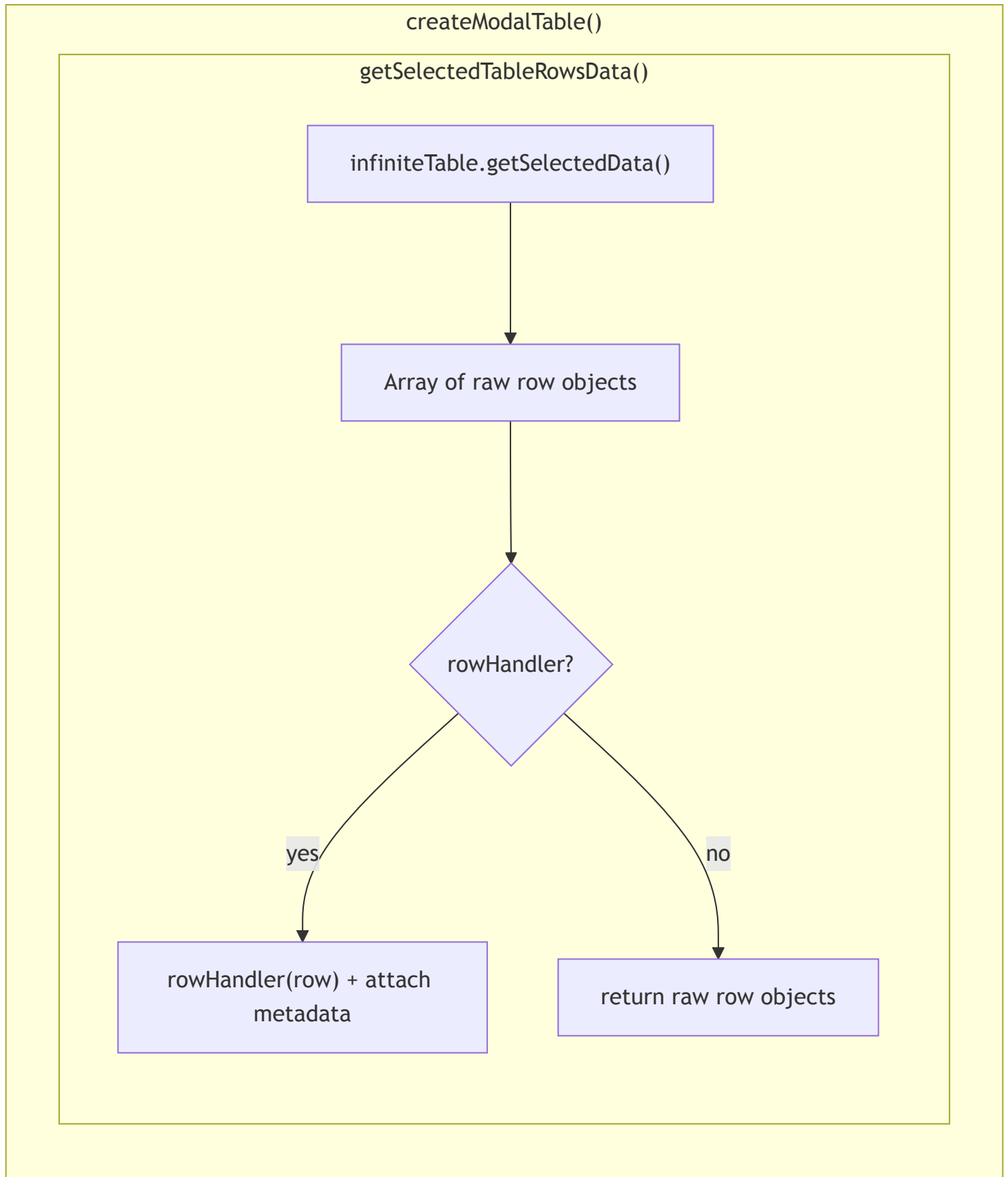


8. GenericDataSource — Data Loading Pipeline (Unchanged)

GenericDataSource is the same data layer from data-modal. It is not coupled to DataTables or InfiniteTable — it produces plain arrays and column lists that either can consume.



9. createModalTable Internals — What InfiniteTable Actually Does



buildTable() – lazy, runs
once per datasource

`datasource.tableData()`



`datasource.tableColumns()`



`createInfiniteTable({container,
columns, columnDefs,
selectionStyle})`



`infiniteTable.setRowHandler(datasource.rowHandler)`



`infiniteTable.setData(tableData)`



VirtualScroller renders
visible rows only

Factory Construction

```
createModalElement(id,  
title, description)
```

```
parentElement.appendChild(modalElement)
```

```
new  
bootstrap.Modal(modalElement)
```

```
addEventListener('shown.bs.modal',  
buildTable)
```

```
okButton.addEventListener('click',  
handleOK)
```

InfiniteTable Features vs DataTables Features

Feature	DataTables (old)	InfiniteTable (new)
Row display	Pagination (<code>pageLength: 100</code>)	Virtual scrolling (all rows available, only visible rendered)
Row selection	DataTables Select extension + <code>.selected</code> CSS class	<code>createRowSelection</code> with <code>Set<index></code> state
Search/filter	DataTables built-in search bar (present but unused)	<code>createSearchFilter</code> with debounced real-time search (functional)
Horizontal scroll	<code>scrollX: true</code>	CSS grid with <code>minmax(120px, 1fr)</code> columns
Vertical scroll	<code>scrollY: '400px'</code>	Virtual scroller — fixed-height body with O(1) row lookup
Column titles	<code>columns: [{title, data}]</code> config	<code>columnDefs: {col: {title: 'Display Name'}}</code>
DOM weight	All rows rendered + pagination controls	Only visible + buffer rows (~40-50 DOM nodes)
Dependencies	jQuery 3.5.1 + DataTables + Select extension (3 CDN scripts + 1 CDN CSS)	Zero external dependencies
Data extraction	<code>api.row(tr).index()</code> → <code>tableData[index]</code>	<code>selection.getSelectedIndices()</code> → <code>data[index]</code>

10. Row Selection and Data Extraction — Detail

Same logical flow as before, but the mechanism changes from jQuery DOM queries to a `Set`-based index tracker:



```

    └─ multi mode: selected.has(originalIndex) ? delete : add
    |
    ▼
onSelectionChange → scroller.refresh() → re-render with .infinite-table__row--selected
    |
    ▼ User clicks OK button
    |
getSelectedTableRowsData()
    |
    └─ infiniteTable.getSelectedData()
    |   selection.getSelectedIndices() → sorted array of indices
    |   indices.map(i => data[i]) → raw row objects
    |
    └─ If rowHandler exists (ENCODE):
    |   rowHandler(row) → {name, url, color}
    |   Attach metadata: filtered columns from raw row
    |   Return: [{name, url, color, metadata: {Biosample, Target, ...}}, ...]
    |
    └─ If no rowHandler (GenArk, custom):
    |   Return: [{accession, assembly, scientificName, ...}, ...]
    |
    ▼
okHandler(result)
    |
    └─ trackLoadHandler → browser.loadTrackList(result) [ENCODE, custom]
    └─ loadGenome({genarkAccession: result[0].accession}) [GenArk]

```

11. Modal DOM Structure

createModalTable generates this Bootstrap modal HTML. InfiniteTable renders inside `#${id}-table-container`.

```

<div id="${id}" class="modal fade">
  <div class="modal-dialog modal-xl">
    <div class="modal-content">

      <div class="modal-header">
        <div class="modal-title">${title}</div>
        <button class="btn-close" data-bs-dismiss="modal"/>
      </div>

      <div class="modal-body">
        <!-- Description slot -->

        <div style="font-size:0.9rem; padding-bottom:0.75rem">
          ${description HTML, e.g. ENCODE link}
        </div>

        <!-- Spinner (hidden by default) -->
        <div id="${id}-spinner" class="spinner-border" style="display:none"/>

        <!-- InfiniteTable renders here -->
        <div id="${id}-table-container">
          <div class="infinite-table">
            <div class="infinite-table__header">
              <input class="infinite-table__search" placeholder="Search..." />
              <div class="infinite-table__header-row" style="display:grid">
                <!-- column header cells -->
              </div>
            </div>

```

```

</div>
<div class="infinite-table__body" style="overflow-y:auto; height:400px">
  <div class="infinite-table__sentinel" style="height: totalRows x rowHeight">
    <div class="infinite-table__visible" style="transform: translateY(...)">
      <!-- only visible + buffer rows rendered -->
      <div class="infinite-table__row" data-index="...">
        <!-- cells via CSS grid -->
      </div>
    </div>
  </div>
</div>
</div>
</div>
</div>
</div>
</div>

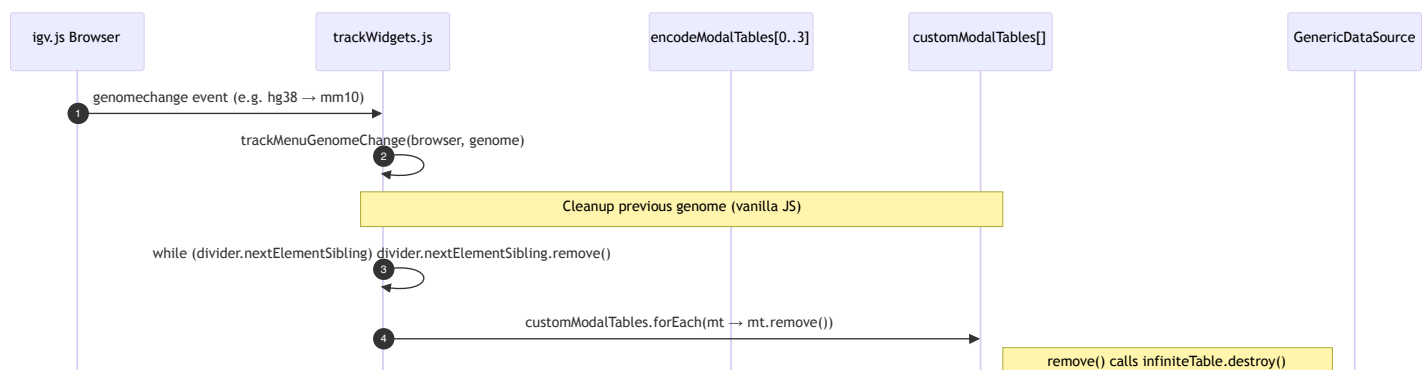
<div class="modal-footer">
  <button data-bs-dismiss="modal">Cancel</button>
  <button data-bs-dismiss="modal">OK</button>    ← triggers getSelectedTableRowsData
</div>

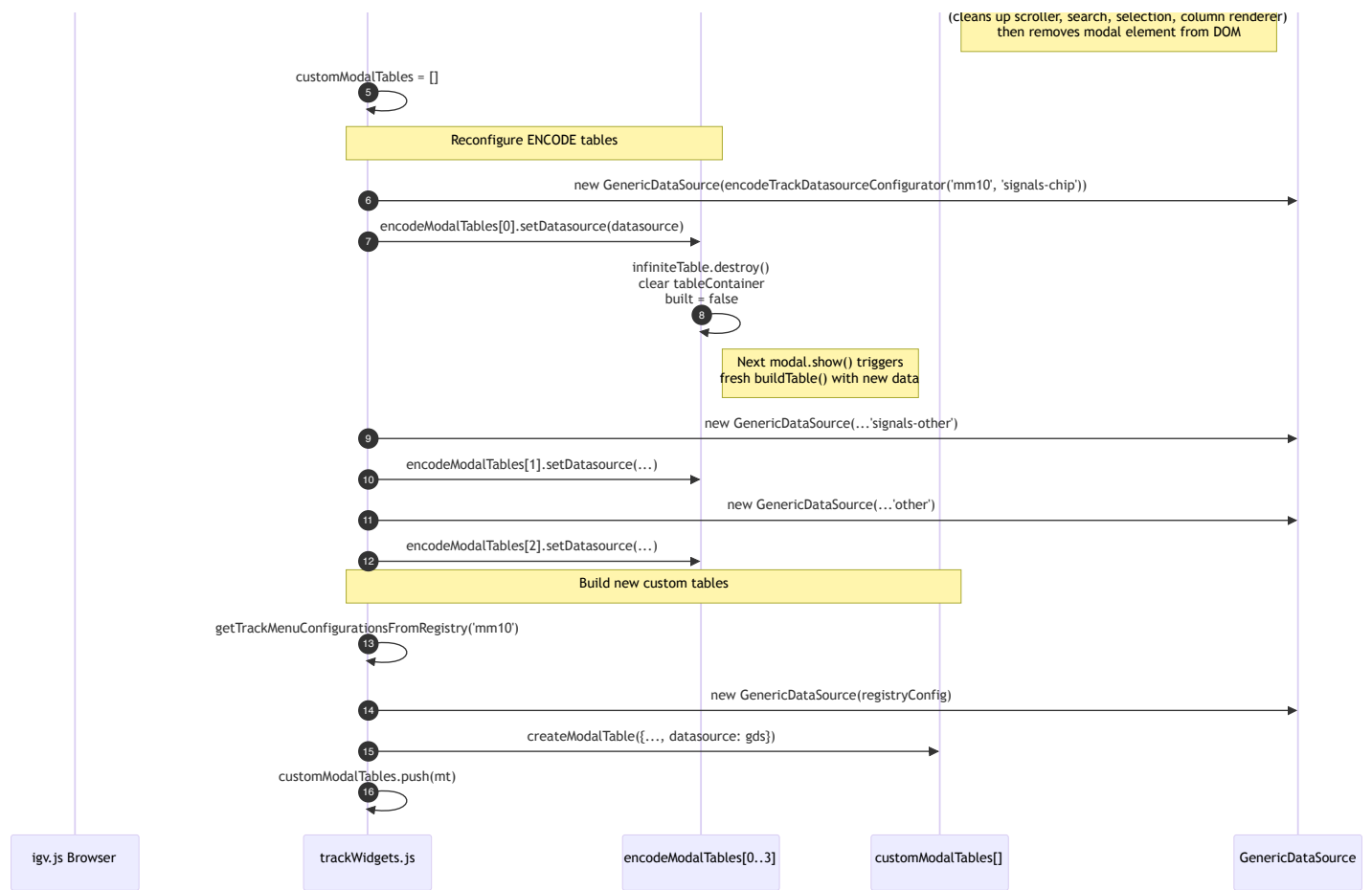
</div>
</div>
</div>

```

12. Genome Change — DataSource Swap

Same sequence as before, but with InfiniteTable teardown/rebuild instead of DataTables:





13. Component Responsibilities

Component	Location	Responsibi
createModalTable	infinite-table/src/modalTable.js	Bootstrap modal wrap InfiniteTable lifecycle selection extraction + dispatch
createInfiniteTable	infinite-table/src/infiniteTable.js	Composes scroller/search/select manages data and dis
createVirtualScroller	infinite-table/src/virtualScroller.js	DOM pool recycling, s height, visible range c rAF throttle
createSearchFilter	infinite-table/src/searchFilter.js	Debounced text input computed search stri based filtering
createRowSelection	infinite-table/src/rowSelection.js	Set-based single/mul click handling, selecti
		CSS grid layout, colu

createColumnRenderer	infinite-table/src/columnRenderer.js	CSS grid layout, color overrides, row render
GenericDataSource	data-modal/src/genericDataSource.js	Data loading (URL/inli parsing (TSV/CSV/JSO filtering, sorting, row transformation
encodeTrackDatasourceConfigurator	js/widgets/encodeTrackDatasourceConfigurator.js	ENCODE-specific cor columns, rowHandler,
genarkDatasourceConfigurator	js/widgets/genarkDatasourceConfigurator.js	GenArk-specific conf URL, columns, custom
trackWidgets.js	js/widgets/trackWidgets.js	Creates/manages EN custom modal tables, dropdown DOM, wire: to <code>browser.loadTrack</code>
genomeWidgets.js	js/widgets/genomeWidgets.js	Creates/manages Ger table, wires okHandle <code>loadGenome()</code>
trackSelectionModal.js	js/widgets/trackSelectionModal.js	Non-InfiniteTable che (unchanged)
trackSelectionListModal.js	js/widgets/trackSelectionListModal.js	Non-InfiniteTable sele (unchanged)

14. What Changed vs What Stayed

REPLACED

```

| ModalTable class |
| → createModalTable |
|
| DataTables API |
| → InfiniteTable |
| (virtual scroll) |
|
| jQuery |
| → vanilla JS |
|
| CDN dependencies |
| (3 scripts, 1 CSS) |
| → 0 CDN, 1 local |
| CSS |

```

KEPT AS-IS

```

| GenericDataSource |
| • data loading |
| • parsing |
| • filtering/sorting |
| • rowHandler |
| • tableColumns() |
| • tableData() |
|
|
| Datasource configs |
| • encodeTrack... |
| • genark... |
| • registry JSON files |
|
|
| Consumer code shape |
| • trackWidgets.js |
| • genomeWidgets.js |
| (okHandler wiring, |
| setDatasource calls) |

```

15. Side-by-Side Comparison: DataTables vs InfiniteTable

API Mapping

Operation	DataTables (old)	InfiniteTable (new)
Create table instance	<code>new ModalTable(config)</code>	<code>createModalTable(config)</code>
Config: pagination	<code>pageLength: 100</code>	<i>(removed — virtual scrolling replaces pagination)</i>
Config: selection mode	<code>selectionStyle: 'single'</code>	<code>selectionStyle: 'single'</code> (same)
Swap data source	<code>mt.setDatasource(ds)</code>	<code>mt.setDatasource(ds)</code> (same)
Update description	<code>mt.setDescription(html)</code>	<code>mt.setDescription(html)</code> (same)
Remove from DOM	<code>mt.remove()</code>	<code>mt.remove()</code> (same)
Show modal	<code>mt.modal.show()</code>	<code>mt.modal.show()</code> (same)
Get selected data	<i>internal via</i> <code>\$('#tr.selected')</code>	<i>internal via</i> <code>infiniteTable.getSelectedData()</code>

Dropdown DOM Manipulation (trackWidgets.js)

Operation	jQuery (old)	Vanilla JS (new)
Find dropdown menu	<code>\$('#igv-app-track-dropdown-menu')</code>	<code>document.getElementById('igv-app-track-dropdown-menu')</code>
Find divider	<code>\$dropdownMenu.find('#igv-app-annotations-section')</code>	<code>document.getElementById('igv-app-annotations-section')</code>
Remove siblings	<code>\$divider.nextAll().remove()</code>	<code>while (divider.nextElementSibling) divider.nextElementSibling.remove()</code>
Create button	<code>\$('#<button>', {class:..., type:...})</code>	<code>document.createElement('button')</code> + property assignment
Insert after	<code>\$button.insertAfter(\$divider)</code>	<code>divider.insertAdjacentElement('afterend', button)</code>
Click handler	<code>\$button.on('click', fn)</code>	<code>button.addEventListener('click', fn)</code>

Performance Characteristics

Aspect	DataTables	InfiniteTable
DOM nodes for 10,000 rows	~10,000 <code><tr></code> elements (paginated view shows 100)	~40–50 <code><div></code> elements (visible + buffer)

Initial render	Renders all rows into DOM, then paginates	Renders only visible viewport
Search	DataTables built-in (full re-render)	Pre-computed lowercase strings, index-based filter, no DOM rebuild
Scroll performance	Native table scroll (all rows in DOM)	requestAnimationFrame + DOM node recycling
Memory	Full DOM tree + DataTables internal state	Minimal DOM + flat arrays + Set for selection

CDN / Bundle Impact

Resource	DataTables (old)	InfiniteTable (new)
jQuery 3.5.1 slim	70 KB min	Removed
DataTables 1.10.20	80 KB min	Removed
DataTables Select 1.3.1	15 KB min	Removed
DataTables CSS	8 KB	Removed
InfiniteTable JS	—	~6 KB (bundled into app via rollup)
InfiniteTable CSS	—	~2 KB
Net change	~173 KB CDN	~8 KB local (bundled)